

Real-world usage of SparkSQL

Sure! Let's dive into the concept of **Spark SQL** in simple terms.

Understanding Spark SQL: A Beginner's Guide

Spark SQL is a powerful tool that allows you to work with structured data using SQL (Structured Query Language) within the Spark framework. Think of it as a bridge that connects the world of traditional databases with the fast and scalable processing capabilities of Spark. With Spark SQL, you can run SQL queries on data stored in Spark DataFrames, which are like tables in a database but designed for big data processing.

Creating Table Views

To use Spark SQL, the first step is to create a "table view." Imagine a table view as a temporary window into your data. You can create two types of views: a temporary view, which is like a local window that only you can see during your current session, and a global temporary view, which is like a shared window that everyone can access across different sessions. This allows you to run SQL queries and analyze your data easily.

If you want to create a temporary view in Python using PySpark, you would first load your data into a DataFrame and then create the view. Here's a simple example:

```
# Create a DataFrame from a JSON file
df = spark.read.json("people.json")

# Create a temporary view called "people"
df.createOrReplaceTempView("people")
```

This code snippet shows how to load data from a JSON file and create a temporary view named "people." Now, you can run SQL queries on this view to analyze your data!

Spark SQL - revisited

- Is a Spark module for structured data processing
- Runs SQL queries on Spark DataFrames
- Usable in Java, Scala, Python and R

Creating a View in Spark SQL

- Creating a table view in Spark SQL is required to run SQL queries programmatically on a DataFrame
- A view is a temporary table to run SQL queries
 - A Temporary view provides local scope within the current Spark session
 - A Global Temporary view provides global scope within the Spark application

Creating a View in Spark SQL

```
# Create DataFrame from file
df =
spark.read.json("people.json")

# Create a temp view
df.createTempView("people")
# Run SQL Query
spark.sql("SELECT * FROM
people").show()
```

```
+-----+-----+
|  age | name  |
+-----+-----+
| null | Michael |
|  30  |  Andy  |
|  19  |  Justin |
+-----+-----+
```

Creating a View in Spark SQL

```
# Create a global view
df.createGlobalTempView("people")
# Run SQL Query
spark.sql("SELECT * FROM
global_temp.people").show()
```

```
+-----+-----+
|  age | name  |
+-----+-----+
| null | Michael |
|  30  |  Andy  |
|  19  |  Justin |
+-----+-----+
```

- The first step to running SQL queries in Spark SQL is to create a table view. A table view is a temporary table used to run SQL queries. Spark SQL supports both temporary and global temporary table views. A temporary view has local scope. Local scope means that the view exists only within the current spark session on the current node. A global temporary view, however, exists within the general Spark application. A global temporary view is shareable across different Spark sessions.

- Here is an example that shows how to create a temporary view in Python using PySpark. You first create the DataFrame from the JSON file, then create a temporary view called "people." You can then run a SQL query using this view.
- Here is the same example, but this time, you are creating a Global Temporary View. Note the minor syntax change, including the "Global" prefix to the function name and the "global temp" prefix to the view name.

Aggregating Data

Used to aggregate data over columns

- DataFrames contain inbuilt common aggregation functions - `count()`, `countDistinct()`, `avg()`, `max()`, `min()`, and others
- Alternatively, aggregate using SQL queries and tableviews

Aggregating Data - An Example

```
import pandas as pd
mtcars = pd.read_csv('mtcars.csv')
sdf = spark.createDataFrame(mtcars)
```

Aggregating data - An Example

```
sdf.select('mpg').show(5)
```

```
[, 1] mpg Miles/(US) --> gallon
[, 2] cyl -> Number of cylinders
[, 3] disp -> Displacement (cu.in.)
[, 4] hp -> Gross horsepower
[, 5] drat --> Rear axle ratio
```

Aggregating data – An Example

```
sdf.select('mpg').show(5)
```

```
+-----+  
|  mpg  |  
+-----+  
| 21.0  |  
| 21.0  |  
| 22.8  |  
| 21.4  |  
| 18.7  |  
+-----+
```

only showing top 5 rows

Aggregating data - An Example

```
# Using a DataFrame function
```

```
car_counts =  
sdf.groupby(['cyl']).agg({"wt": "count"}).sort("count(wt)",  
ascending=False).show(5)
```

```
# Using an SQL Query + Table View
```

```
sdf.createTempView("cars")  
sql("SELECT cyl, COUNT(*) FROM cars GROUPBY cyl ORDER by 2  
DESC")
```

- Aggregation, a standard Spark SQL process, is generally used to present grouped statistics. DataFrames come inbuilt with commonly used aggregation functions such as count, average, max, and others. You can also perform aggregation programmatically using SQL queries and table views.
- First, import your data into a DataFrame. Then, use pandas on Python to read the CSV file and create a DataFrame. You can apply the Select function to examine data from a specific column in detail. This example applies the select function to view the first five rows of the "mpg" column.

- Here, you can see the first few rows of the dataset.
- In this example, you aggregate and group cars by the number of cylinders they have. You can perform this action using two methods: The first method is to use the inbuilt functions of the DataFrame. You have already covered this operation in another lesson. The second method is to create a temp view for the DataFrame. In this example, you name the temp view "cars." Then, you run a SQL query to group by cylinders and view the data in descending order. Although you use different tools and functions, both approaches produce the same results.

Spark SQL Data Sources

- **Parquet Files**
 - Supports reading/ writing and preserving data schema.
 - Spark SQL can also run queries without loading the file
 - **JSON Datasets:** Spark infers the schema and loads the dataset as a DataFrame
 - **Hive Tables:** Spark supports reading and writing data stored in Apache Hive
-
- Next, let's look at some of the data sources that Spark SQL supports. Parquet is a columnar format supported by many data processing systems. Spark SQL supports reading and writing data from Parquet files, and Spark SQL preserves the data schema. Similarly, Spark SQL can load and write to JSON datasets by inferring the schema. Spark SQL also supports reading and writing data stored in Hive.

Summary

In this video, you learned that:

- Spark modules for structured data processing can run SQL queries on Spark DataFrames and are usable in Java, Scala, Python and R
- Spark SQL supports both temporary views and global temporary views
- You can use a DataFrame function or an SQL Query + Table View for data aggregation
- Spark SQL supports Parquet files, JSON datasets and Hive tables